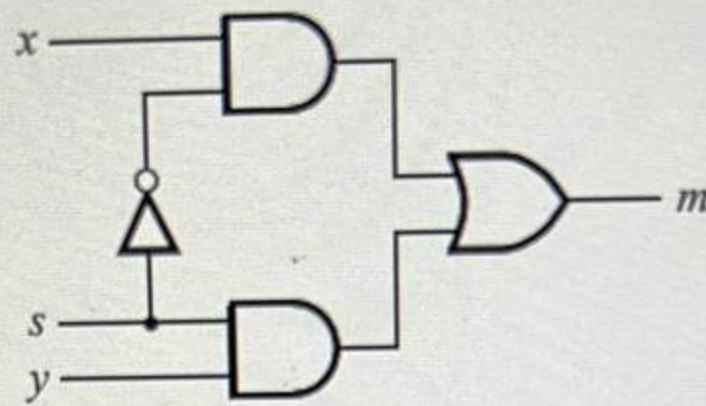


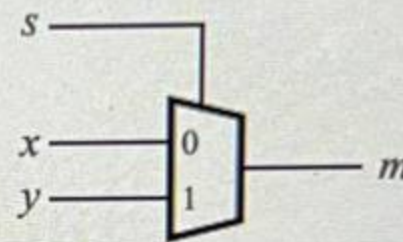
ows a sum-of-products circuit that implements a 2-to-1 *multiplexer* with a select input s . If $s = 0$, the output m is equal to the input x , and if $s = 1$ the output is equal to y . Part *b* of the figure shows the truth table for this multiplexer, and part *c* shows its circuit symbol.



a) Circuit

s	m
0	x
1	y

b) Truth table



c) Symbol

Figure 2: A 2-to-1 multiplexer.

A multiplexer can be described by the following VHDL statement:

```
m <= (NOT (s) AND x) OR (s AND y);
```


0 then $M = X$, while $s = 1$ then $M = Y$. We refer to this circuit as a four-bit wide 2-to-1 multiplexer, as the circuit symbol shown in Figure 3b, in which X , Y , and M are depicted as four-bit wires.

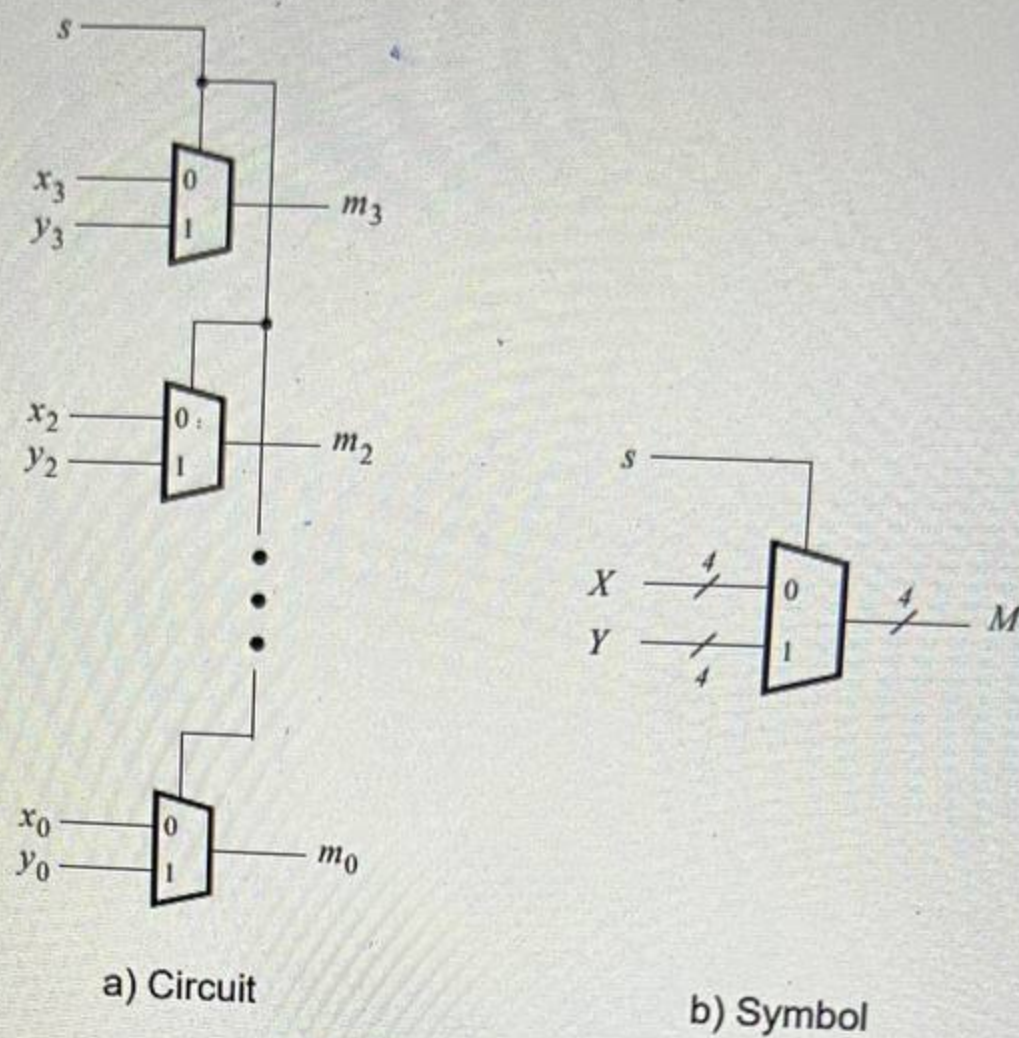


Figure 3: A four-bit wide 2-to-1 multiplexer.

perform the steps listed below.

1. Create a new Quartus project for your circuit.
2. Include your VHDL file for the four-bit wide 2-to-1 multiplexer in your project. Use switch SW_7-4 as the s input, switches SW_3-0 as the X input and SW_7-4 as the Y input. Display the value of the input s on the LEDR lights, and connect the output M to $LEDR_3-0$, and connect the unused LEDR lights to the constant value 0.


```
=====
part 2
-----
LIBRARY ieee;
USE ieee.std_logic_1164.all;

-- Simple entity that connects the SW switches to the LEDR lights
ENTITY part_one IS

PORT (  SW   : IN  STD_LOGIC_VECTOR(9 DOWNTO 0);
       LEDR  : OUT STD_LOGIC_VECTOR(9 DOWNTO 0));

END part_one;

ARCHITECTURE structure OF part_one IS

signal sel : std_logic;
signal X, Y ,M : std_logic_vector(3 DOWNTO 0);

BEGIN

X <= sw(3 downto 0);
y <= sw(7 downto 4);

sel <= sw(9);

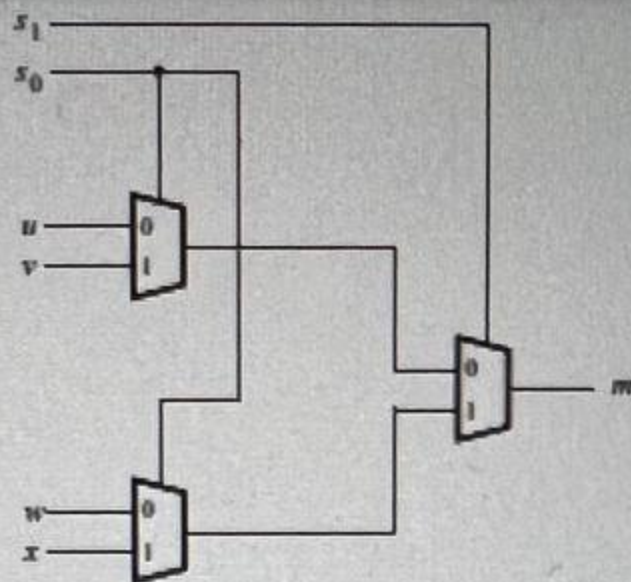
m(0) <= (not (sel) and X(0)) or (sel and y(0));
m(1) <= (not (sel) and X(1)) or (sel and y(1));
m(2) <= (not (sel) and X(2)) or (sel and y(2));
m(3) <= (not (sel) and X(3)) or (sel and y(3));

ledr(9) <= sel;

ledr(8 downto 4) <= "00000";
ledr(3 downto 0) <= M;

END structure;

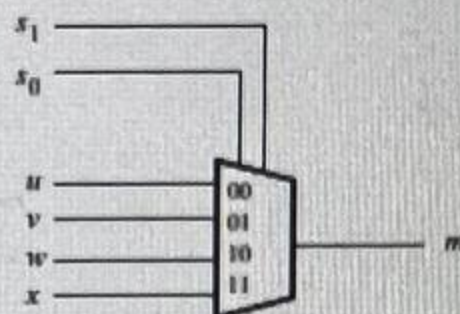
=====
part 3
-----
```

a) Circuit

s_1	s_0	m
0	0	u
0	1	v
1	0	w
1	1	x

b) Truth table



c) Symbol

Figure 4: A 4-to-1 multiplexer.

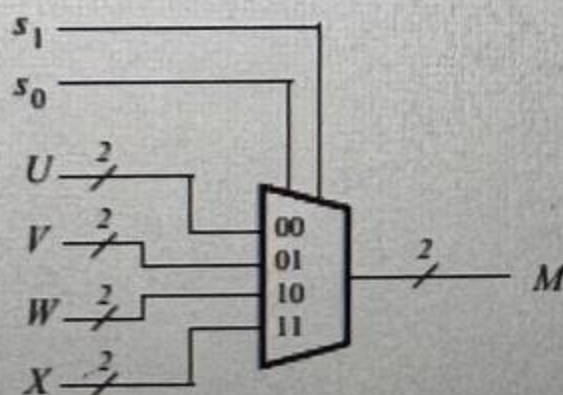


Figure 5: A two-bit wide 4-to-1 multiplexer.

form the following steps to implement the two-bit wide 4-to-1 multiplexer.

1. Create a new Quartus project for your circuit.
2. Create a VHDL entity for the two-bit wide 4-to-1 multiplexer. Connect its select inputs to switches SW_{9-8} , and use switches SW_{7-0} to provide the four 2-bit inputs U to X . Connect the output M to the red lights $LEDR_{1-0}$.
3. Include in your project the required pin assignments for your DE-series board. Compile the project.

=====

part 3

LIBRARY ieee;

USE ieee.std_logic_1164.all;

ENTITY part_one IS

PORT (SW : IN STD_LOGIC_VECTOR(9 DOWNT0 0);

LEDR : OUT STD_LOGIC_VECTOR(9 DOWNT0 0))

END part_one;

ARCHITECTURE structure OF part_one IS

signal U , V , W ,X , sel , M : std_logic_vector(1 DOWNT0 0);

BEGIN

U <= sw(7 downto 6);

V <= sw(5 downto 4);

W <= sw(3 downto 2);

X <= sw(1 downto 0);

sel <= sw(9 downto 8);

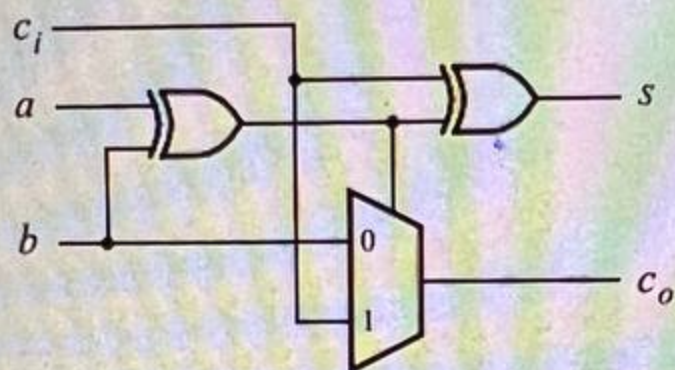
M <= u when sel="00" else
v when sel="01" else
w when sel="10" else
x when sel="11" else
"00";

LEDR(1 downto 0) <= M ;

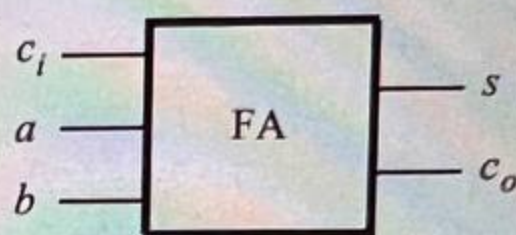
END structure;

<

way that the carry signals are passed from one full adder to the next. Write VHDL code that implements it, as described below.



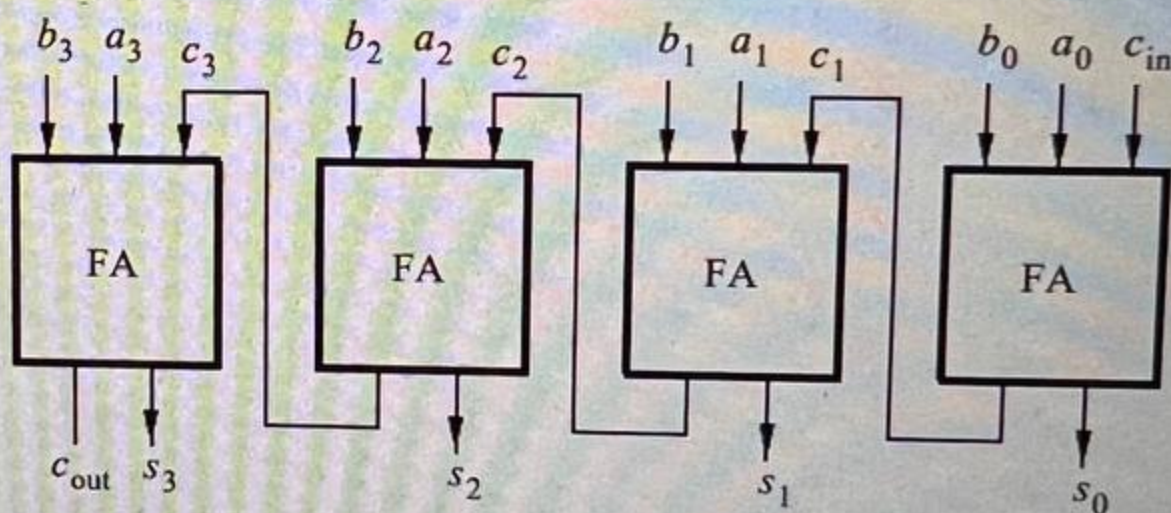
a) Full adder circuit



b) Full adder symbol

b	a	c_i	c_o	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

c) Full adder truth table



d) Four-bit ripple-carry adder circuit

Figure 2: A ripple-carry adder circuit.

1. Create a new Quartus project for the adder circuit. Write a VHDL entity for the full adder subcircuit and write a top-level VHDL entity that instantiates four instances of this full adder.
2. Use switches SW_{7-4} and SW_{3-0} to represent the inputs A and B , respectively. Use SW_8 for the carry.

```
library ieee ;  
use ieee.std_logic_1164.all;
```

```
entity full_adder is  
    port(  
        a,b,c_in : in std_logic;  
        c_out ,s : out std_logic  
    );  
end full_adder;
```

```
architecture structure of full_adder is
```

```
    signal sig : std_logic;
```

```
begin
```

```
    sig <= a xor b ;  
    s  <= sig xor c_in ;
```

```
    c_out <= b    when sig = '0' else  
             c_in    when sig = '1';
```

```
end structure;
```

```
--=====+++++
```



```
library ieee ;  
use ieee.std_logic_1164.all;
```

```
entity lab2_part3 is  
    port(  
        swX  : in  std_logic_vector(9 downto 0);  
        ledrX : out std_logic_vector(4 downto 0)  
    );  
end lab2_part3;
```

architecture structure of lab2_part3 is

```
    COMPONENT full_adder  
        PORT (  
            a , b ,c_in : in  std_logic;  
            s , c_out  : out std_logic  
        );  
    END COMPONENT;
```

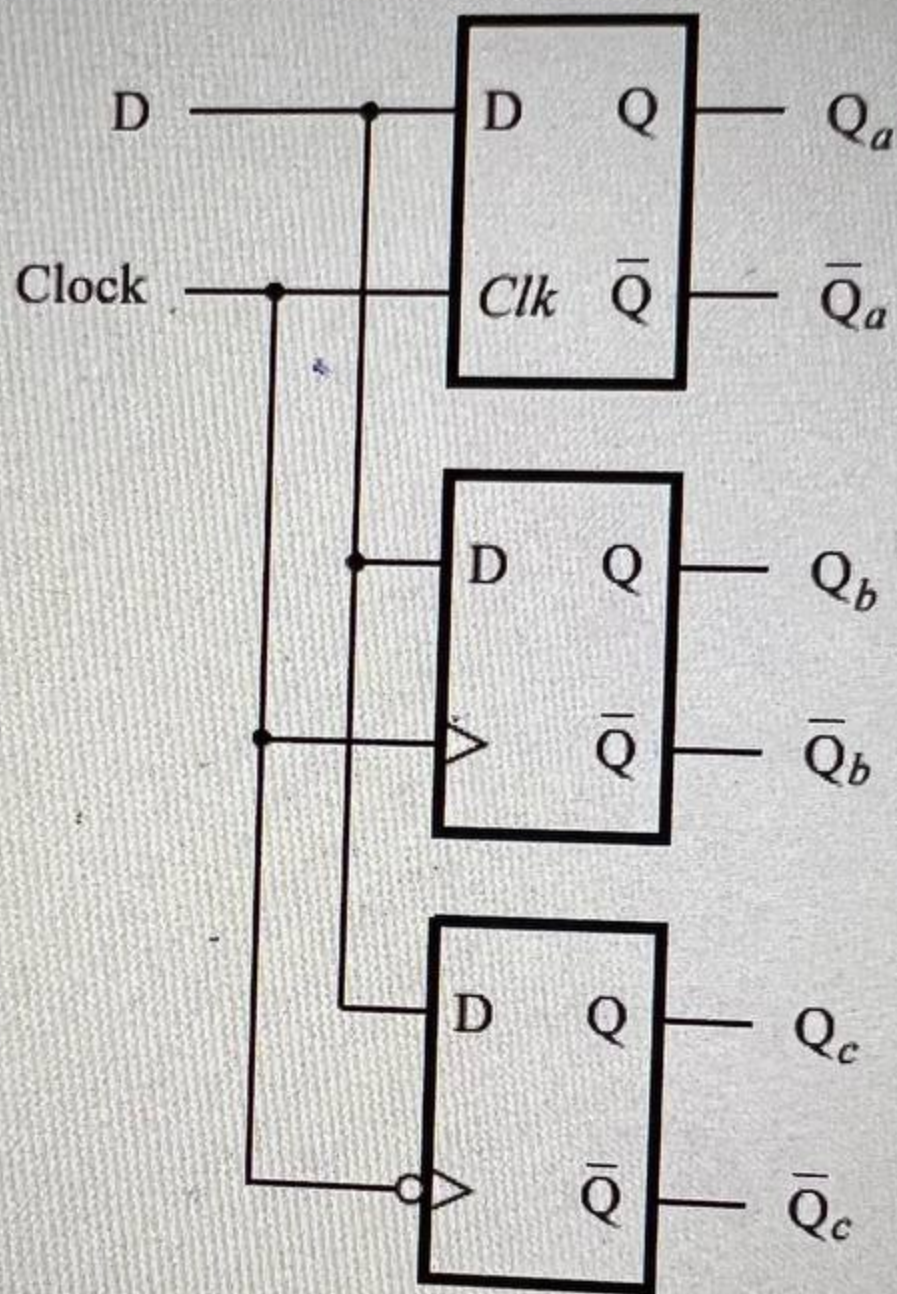
```
    signal c1,c2,c3: std_logic;
```

```
begin
```

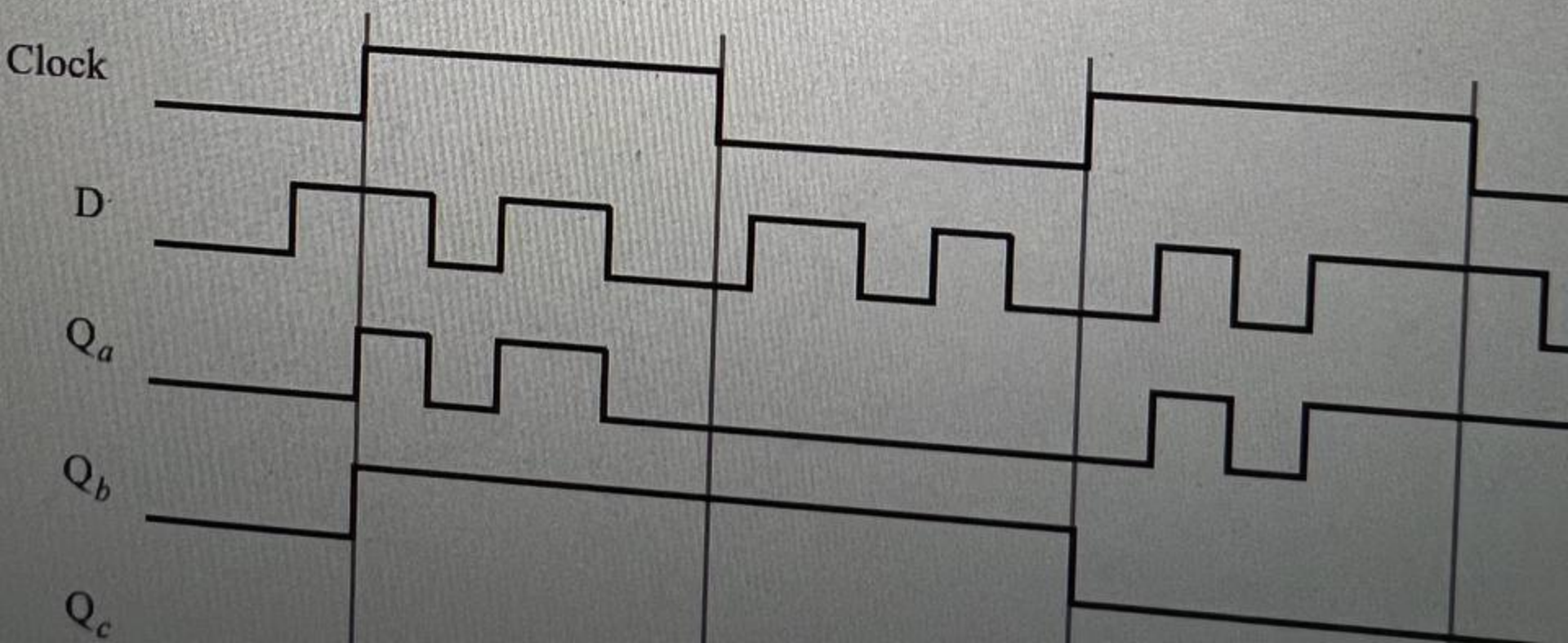
```
fa1: full_adder port map( a => swX(0) , b => swX(4) , c_in =>  
    swX(8), s => ledrX(0) , c_out => c1    );  
fa2: full_adder port map( a => swX(1) , b => swX(5) , c_in =>  
    c1  , s => ledrX(1) , c_out => c2    );  
fa3: full_adder port map( a => swX(2) , b => swX(6) , c_in =>  
    c2  , s => ledrX(2) , c_out => c3    );  
fa4: full_adder port map( a => swX(3) , b => swX(7) , c_in =>  
    c3  , s => ledrX(3) , c_out => ledrX(4));
```

```
end structure;
```


is a circuit with three different storage elements: a gated D latch, a positive-edge triggered D flip-flop, and a negative-edge triggered D flip-flop.



(a) Circuit



File Edit Format View Help

```
USE ieee.std_logic_1164.all;
```

```
-- inputs:
```

```
-- Clk: manual clock
```

```
-- D: data input
```

```
--
```

```
-- outputs:
```

```
-- Qa: gated D-latch output
```

```
-- Qb: positive edge-triggered D flip-flop output
```

```
-- Qc: negative edge-triggered D flip-flop output
```

```
ENTITY 13p4 IS
```

```
PORT (
```

```
key, sw : IN STD_LOGIC_vector(1 downto 0);
```

```
ledr : OUT STD_LOGIC_vector(2 downto 0)
```

```
);
```

```
END 13p4;
```

```
ARCHITECTURE Behavior OF 13p4 IS
```

```
signal Clk, D : STD_LOGIC;
```

```
signal Qa, Qb, Qc : STD_LOGIC;
```

```
BEGIN
```

```
clk <= not key(0);
```

```
d <= sw(0);
```

```
-- gated D-latch
```

```
PROCESS (D, Clk)
```

```
BEGIN
```

```
IF (Clk = '1') THEN
```

```
Qa <= D;
```

```
END IF;
```

```
END PROCESS;
```

```
-----
```

```
PROCESS (Clk)
```

```
BEGIN
```

```
IF (Clk'EVENT AND Clk = '1') THEN
```

```
Qb <= D;
```

```
END IF;
```

```
END PROCESS;
```

```
-----
```

```
PROCESS (Clk)
```

```
BEGIN
```

```
IF (Clk'EVENT AND Clk = '0') THEN
```

```
Qc <= D;
```

```
END IF;
```

```
END PROCESS;
```

```
ledr(0) <= qa;
```

```
ledr(1) <= qb;
```

```
ledr(2) <= qc;
```

```
END Behavior;
```

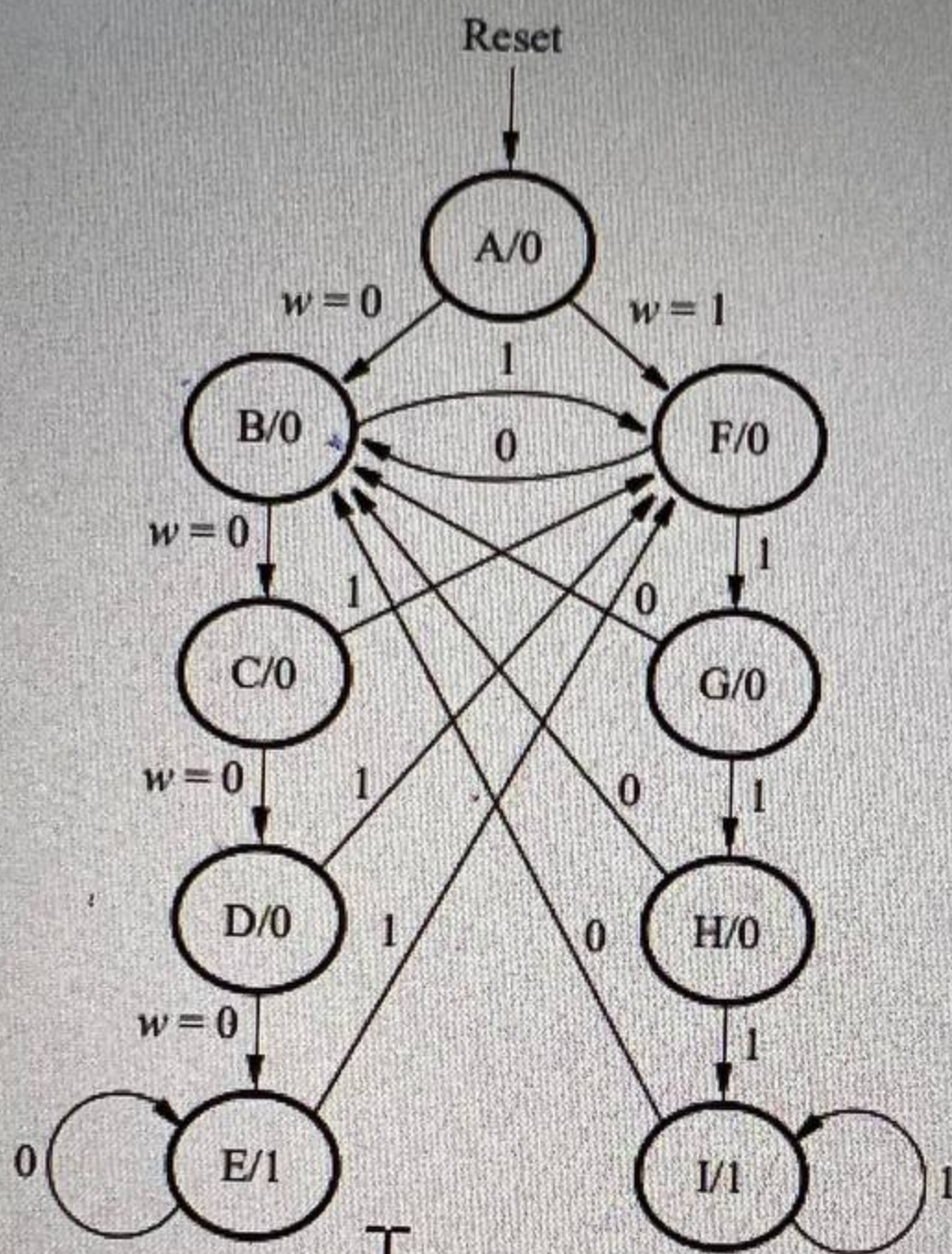



Figure 2: A state diagram for the FSM.

Name	State Code								
	y_8	y_7	y_6	y_5	y_4	y_3	y_2	y_1	y_0
A	0	0	0	0	0	0	0	0	1
B	0	0	0	0	0	0	0	1	0
C	0	0	0	0	0	0	1	0	0
D	0	0	0	0	0	1	0	0	0
E	0	0	0	0	1	0	0	0	0
F	0	0	0	1	0	0	0	0	0
G	0	0	1	0	0	0	0	0	0
H	0	1	0	0	0	0	0	0	0
I	1	0	0	0	0	0	0	0	0


```

LIBRARY ieee;
USE ieee.std_logic_1164.all;

-- A sequence detector FSM
-- SW0 is the active low synchronous reset, SW1 is the w input, and KEY0 is the clock.
-- The z output appears on LEDG0, and the state is indicated on LEDR8..0
ENTITY part2 IS
    PORT (
        SW      : IN  STD_LOGIC_VECTOR(1 DOWNTO 0);
        KEY     : IN  STD_LOGIC_VECTOR(0 DOWNTO 0);
        LEDG    : OUT STD_LOGIC_VECTOR(0 DOWNTO 0);
        LEDR    : OUT STD_LOGIC_VECTOR(8 DOWNTO 0));
END part2;

ARCHITECTURE Behavior OF part2 IS
    SIGNAL Clock, Resetn, w, z : STD_LOGIC;
    TYPE State_type IS (A, B, C, D, E, F, G, H, I);
    SIGNAL y_Q, Y_D : State_type;
BEGIN
    Clock <= KEY(0);
    Resetn <= SW(0);
    w <= SW(1);

    PROCESS (w, y_Q) -- state table
    BEGIN
        CASE y_Q IS
            WHEN A => IF (w = '0') THEN Y_D <= B;
                       ELSE Y_D <= F;
                       END IF;
            WHEN B => IF (w = '0') THEN Y_D <= C;
                       ELSE Y_D <= F;
                       END IF;
            WHEN C => IF (w = '0') THEN Y_D <= D;
                       ELSE Y_D <= F;
                       END IF;
            WHEN D => IF (w = '0') THEN Y_D <= E;
                       ELSE Y_D <= F;
                       END IF;
            WHEN E => IF (w = '0') THEN Y_D <= E;
                       ELSE Y_D <= F;
                       END IF;
            WHEN F => IF (w = '0') THEN Y_D <= B;
                       ELSE Y_D <= G;
                       END IF;
            WHEN G => IF (w = '0') THEN Y_D <= B;
                       ELSE Y_D <= H;
                       END IF;
            WHEN H => IF (w = '0') THEN Y_D <= B;
                       ELSE Y_D <= I;
                       END IF;
            WHEN I => IF (w = '0') THEN Y_D <= B;
                       ELSE Y_D <= I;
                       END IF;
            WHEN OTHERS => Y_D <= A;
        END CASE;
    END PROCESS; -- state_table

    PROCESS (Clock)
    BEGIN
        IF (Clock'EVENT AND Clock = '1') THEN
            IF (Resetn = '0') THEN -- synchronous clear
                y_Q <= A;
            ELSE
                y_Q <= Y_D;
            END IF;
        END IF;
    END PROCESS;

```

```

END PROCESS;

z <= '1' WHEN ((y_Q = E) OR (y_Q = I)) ELSE '0';
LEDG(0) <= z;

PROCESS (y_Q) -- drive the red LEDs for each state
BEGIN
    LEDR <= "000000000";
    case y_Q IS
        WHEN A => LEDR <= "000000001";
        WHEN B => LEDR <= "000000010";
        WHEN C => LEDR <= "000000100";
        WHEN D => LEDR <= "000001000";
        WHEN E => LEDR <= "000010000";
        WHEN F => LEDR <= "000100000";
        WHEN G => LEDR <= "001000000";
        WHEN H => LEDR <= "010000000";
        WHEN I => LEDR <= "100000000";
    END CASE;
END PROCESS; -- LEDs

END Behavior;

```